Module	Introduction to Image Processing (IIP) / COMP 2032 / Semester 2
Assessment	Coursework (100%)
Module Convenor(s)	Tissa Chandesa

Programming and Conference Paper	**Group**	**50%**

Description and Deliverable(s)

Release Date	Tuesday, 24 February 2026
Submission Date	Friday, 24 April 2026, by 11:59pm
Late Policy **(University of Nottingham default will apply, if blank)**	Work submitted after the deadline will be subject to a penalty of 5 marks (the standard 5% absolute) for each late **working day** out of the total 100 marks.
Feedback Mechanism and Date	Marks and written feedback will be returned via Moodle on the w/c 2 June 2026

AI Declaration
The use of AI in this coursework is **optional**.

The coursework (details below) requires you to **work in groups of 4** to develop a software solution, using Python, to a real image processing problem and collaboratively producing a conference paper describing and critically evaluating your solution. The deliverables required are:

Per group submission:

1. Python codes: *.py* files (submitted as a ZIP folder)
2. A 6-page conference paper in PDF format (**note:** *additional pages will result in a deduction of 5% for every additional page for the entire group*).
3. Results 2026 IIP - Group**XXX** (submitted as a ZIP folder and please change **XXX** to your group number (e.g., 001, 007, 011)
   - 001 – Input Images
   - 002 - Image Processing Pipeline
   - 003 – Output Images

Individual submission:

4. Each of you will need to make an additional submission, **separately** for your **peer assessment**. Please click on the "**Peer Assessment**" link on Moodle to perform this submission. **Note**: *those who fail to submit their peer assessment will be penalised 10% of their individual mark*.
5. Additionally, each of you will need to fill up the provided **AI Declaration** Microsoft Form link on Moodle. Should you *fail to perform a submission, it* **WILL** *result in a penalty of 15% deduction of your individual mark.*

Aside from this **assessment sheet**, you will be provided with the following documentations:

1. **COMP2032-CW-GroupXXX.docx**: this is your conference paper template. Please change **XXX** to your group number (e.g., 001, 007, 011). Also, please **do not** change the formatting and headers. Marks will be deducted if submitted document does not follow the original formatting style. The provided table as well as text coloured red and blue, respectively are to be replaced by your own written words.
2. A compressed **Dataset** folder.
3. A compressed **Ground Truth** folder.
4. A **Software Development Kit (SDK)** black box.

Note: *COMP2032-CW-GroupXXX.docx* **is to be converted to PDF prior to submission and all text should be coloured in black.**

Semantic Segmentation of Blood Cell

Background

The need to separate one material from another, or to identify a particular material, arises in many image processing applications. From industrial inspection, where the task may involve identifying pixels corresponding to correctly manufactured cloth, to human–computer interaction, where detecting human skin is required, the core problem remains the same: developing an image processing pipeline that converts a colour image into a new representation by labelling different materials.

Medical image analysis is an active research area undergoing transformative evolution through the application of image processing and computer vision technologies. In the field of haematology, for example, Ali et al. (2023) state that haematologists bear the burden of labour-intensive manual processes, requiring meticulous examination of countless blood cells under high workloads and frequent distractions to diagnose blood disorders and guide treatment decisions.

However, advancements in automated peripheral blood smear analysis have significantly alleviated these challenges faced by haematologists.

In this coursework, the objective is to develop an **image processing pipeline** to segment pro-myelocytes (PMYs) from their corresponding background. The resulting pipeline will generate relevant output images, which will then be fed into a neural network–based model to automate blood cell classification.

Image Processing Pipeline

An image processing pipeline usually includes at least some of the following steps (but not exhaustively):

Colour space conversion: choose a colour space. Any can be used, but most people choose to work in a lower-dimensional (<3) space whenever possible.

Noise reduction: depending on image quality, some form of noise suppression may be required.

Thresholding/Segmentation: image regions corresponding to blood cells must be identified, and thresholding is a common approach. The method used varies and may be global or local. Methods that automatically determine the threshold value have obvious advantages over those that require user interaction.

Binary image processing: Identifying a perfect threshold value or segmentation method is almost impossible, and most methods will result in some mis-classified pixels. A further binary image processing stage is often needed to clean up the image, hopefully leaving it containing only regions that correspond to required blood cells. Nothing, however, is perfect.

Dataset

You will be using the *Naturalize dataset,* consisting of 16,000 images, separated into 11 classes.

Ground Truth

You will also be provided with 16K images that will be used to fulfil the **SEMANTIC PART** (*more on this, shortly*) of your coursework. Your image processing pipeline should automatically generate **mask** of the PMY as shown in Figure 1, below. To ease the training process, please add "*_mymask*" at the end of each name of the image such as "*image_1_mymask.png*".

Figure 1: An example of a mask created for one PMY.

Following these steps will prepare you to perform the required *training* via Google Colab, later.

*Your **ACTION** as a Group*

SEGMENTATION PART: Design and implement a single **Python program** that inputs one of the images from the provided dataset at a time and ***outputs an output image which has a white background with ONLY the segmented PMY*** as shown in Figure 2**.** You may not need to employ all the steps listed above, but you will probably find it worthwhile to at least consider them all. You are also encouraged to use other strategies covered in this module to address the PMY segmentation problem.

The focus of this coursework is on designing and developing your image processing pipeline to perform the above-mentioned tasks. You must make sure to evaluate the final outcome. The evaluation can be both **qualitative and quantitative** – for example ***mIoU (mean Intersection over Union)*** can be use as the metrics to evaluate your image processing pipeline's performance. More metrics (*that you can use*) for image segmentation will be covered in lecture 7.

You should seek a solution that is as ***automatic*** *as possible* (try to minimise the number of user-supplied parameters) and operates on all the provided images without user intervention (one parameter set for all).

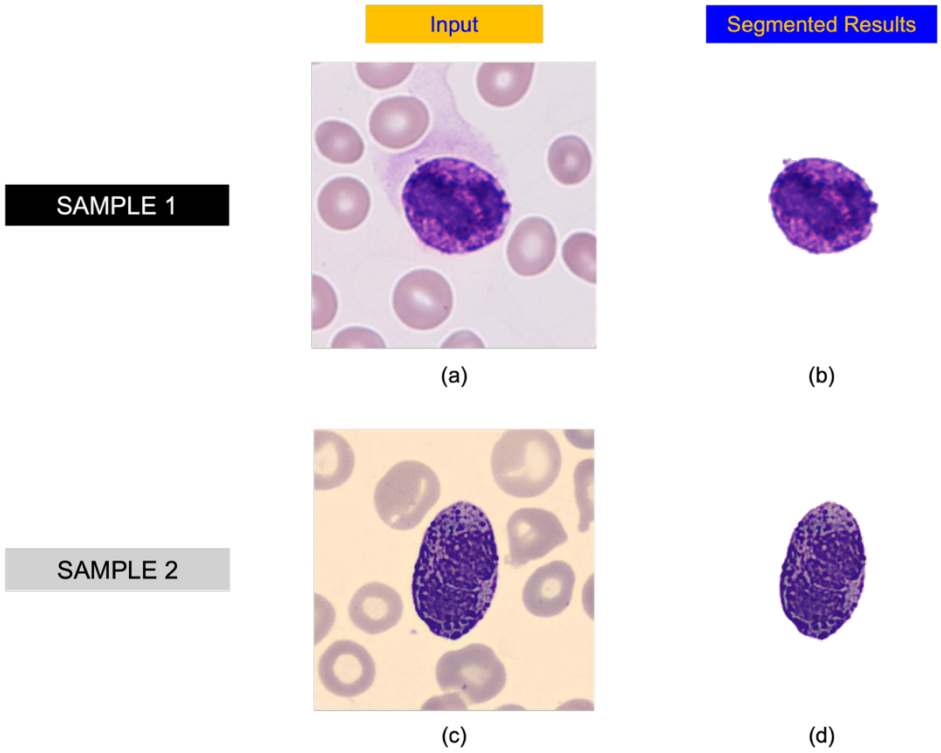

Figure 2: Segmentation Part Desired Outcome

For your conference paper, you should identify 9 images (*3 easy, 3 medium & 3 hard*) to support your arguments of your proposed image processing pipeline.

As for your code, you are required to make your code easy to be executed by the marker. In other words, the marker can run your code and **reproduce** your given outputs. After the execution of your program, it is expected that each image in your provided ***input-images folder*** *(e.g., 9 images – 3 easy, 3 medium & 3 hard)* has a corresponding segmentation result represented by a .jpg file saved in the ***output-images folder***, categorised within three sub-folders (e.g., easy, medium, and hard) with the same name as the input images but with an extension of "*-easy_1, -easy_2, -easy_3, -medium_1, -medium_2, -medium_3, -hard_1, -hard_2 and -hard_3* " at the end of the filename. As mentioned earlier, your output image must have a white background with only the segmented PMY. Additionally, images generated as part of your image processing pipeline can be saved in the ***image-processing-pipeline*** folder whereby you can create sub-folders for each input image and label the sub-folders as the same name as the input image. For everyone convenience, I have provided empty folders/subfolders (e.g., *Results 2026 IIP – Group XXX*, where *XXX* refers to your group number - 001, 007, 011), following the above-mentioned naming convention. All you have to do is fill it up with your desired 9 input images, image processing pipeline images and corresponding output images.

NOTE: A **readme file** is recommended to be produced alongside the source code.

SEMANTIC PART: Once you are satisfied with your image processing pipeline, your final task is to automate blood cell classification. Figure 3 shows the overall process.

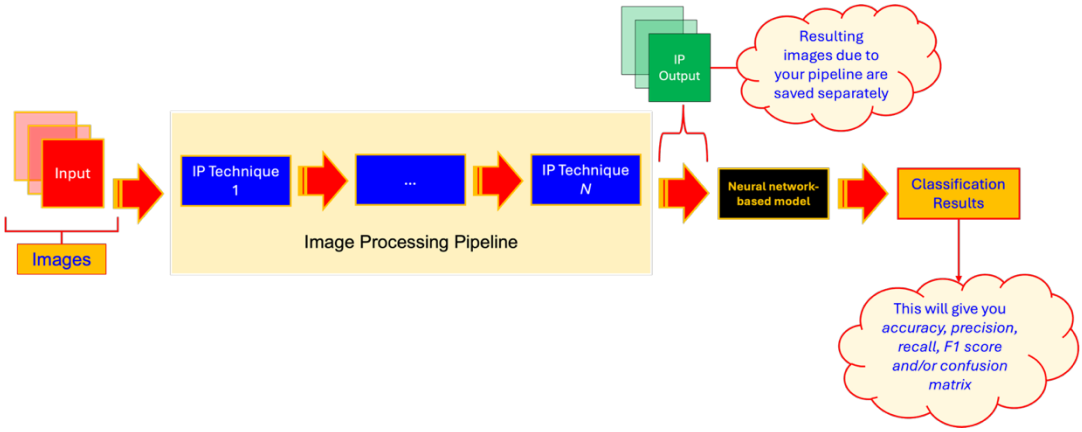

Figure 3: The overall process of a **semantic segmentation**

You have been provided with a black box neural network-based model via an SDK. Algorithm 1 below provide steps on how to use the provided SDK.

```
Step 1: Upload the data and SDK to the Google Colab.

Step 2: !pip install /content/drive/MyDrive/myevalsdk-0.1.2-py3-none-any.whl
Step 3: from myevalsdk import MaskQualityEvaluator
print("Imported successfully")

GT_ROOT   = "/content/drive/MyDrive/testdataset/GT"
PRED_ROOT = "/content/drive/MyDrive/testdataset/student_mask"
IMG_ROOT  = "/content/drive/MyDrive/testdataset/orignal"
ev = MaskQualityEvaluator(
    gt_root=GT_ROOT,
    pred_root=PRED_ROOT,
    image_root=IMG_ROOT,
    output_csv="/content/drive/MyDrive/testdataset/out.csv",
    verbose=True,
)

print("EV ROOTS:")
print("GT :", ev.gt_root)
print("PR :", ev.pred_root)
print("IMG:", ev.image_root)

result_path = ev.run()
print("Saved:", result_path)

```

Algorithm 1: Steps to run the SDK

The classification results will be a **pixel-wise classification** whereby your segmented images (*from the SEGMENTATION PART*) will be classified accordingly to relevant classes. Your **image processing pipeline** is the fundamental component of this entire process whereby your pipeline acts as a way to improve the quality of the dataset prior to being classified.

CONFERENCE PAPER: When writing your conference paper, consider the following points:

1. *Detail* the chosen method(s) and specific image processing technique(s) employed.
2. *Explain* why you choose those technique(s) and method(s). You may include the **pseudocodes** of the chosen technique(s) and method(s) (if applicable). Clearly justify your parameter choices and support your arguments based on either the literature review or trial and error.
3. *Present* the results obtained from your practical implementation.
4. *Critically* evaluate your method(s) on the basis of those results: what are its strengths and weakness? This section should make explicit reference to features of the results you obtained as well as using the relevant metric for segmentation to support your provided arguments/justifications.
5. *Critically* evaluate your classification results: highlighting the strengths and weakness of your entire pipeline by means of performing a pixel-wise classification. Please use the metrics (e.g., *MIoU, Dice and Precision*), generated from your semantic part to further support your provided arguments/justifications in your conference paper.

Reference:

Abou Ali, M., Dornaika, F., & Arganda-Carreras, I. (2023). Blood cell revolution: Unveiling 11 distinct types with 'Naturalize' augmentation. *Algorithms*, *16*(12), 562.

Assessment Criteria

Python code(s): **30%** [**unable to run codes will result in 0% being awarded**]

Conference Paper:

- Justification of chosen algorithms based on performed literature review: **15%**
- Critical analysis and discussion of your findings: **25%**
- Discussion of the strengths and weaknesses of your proposed pipeline (based on your findings), prior to concluding on the classification result: **30%**

In-Lab Test	**Individual**	**50%**

Description	Focuses on theories covered in the lectures and practical exercises
Date, Time & Venue	15 April 2026 **|** 9:30am till 10:30am **|** TCR 2